

UTS
PENGOLAHAN CITRA



NAMA : Muhammad Ridzieq Fachriansyah

NIM : 202331070

KELAS : C

DOSEN : Ir. Darma Rusjdi, M.Kom

NO.PC : 22

ASISTEN : 1. Izzat Islami Kagapi

2. Rizqy Amanda

3. Kashrina Masyid Azka

4. Abdur Rasyid Ridho

INSTITUT TEKNOLOGI PLN
TEKNIK INFORMATIKA
2024/2025

DAFTAR ISI

DAFTAR ISI.....	2
BAB I.....	3
PENDAHULUAN	3
1.1 Rumusan Masalah	3
1.2 Tujuan Masalah	3
1.3 Manfaat Masalah	3
BAB II.....	4
LANDASAN TEORI	4
2.1 Deteksi Warna	4
2.2 Thresholding.....	5
2.3 Histogram Equalization	5
BAB III.....	7
HASIL.....	7
3.1 Deteksi Warna	7
a. Gambar Asli dan Histogram.....	7
b. Deteksi Warna Merah	7
c. Deteksi Warna Biru	8
d. Deteksi Warna Hijau	8
3.2 Nilai Ambang Batas.....	9
3.3 Perbaikan Gambar Backlight	9
a. Menampilkan Citra Asli	9
b. Konversi ke GrayScale	10
c. Meningkatkan Kecerahan.....	10
d. Meningkatkan Kontras	10
e. Kombinasi Kecerahan dan Kontras	11
BAB IV	12
PENUTUP.....	12
4.1 Kesimpulan.....	12
DAFTAR PUSTAKA.....	13

BAB I

PENDAHULUAN

1.1 Rumusan Masalah

1. Bagaimana cara mendeteksi warna biru, merah, dan hijau pada citra?
2. Bagaimana cara menentukan nilai ambang batas (threshold) untuk setiap kategori warna pada citra?
3. Bagaimana cara memperbaiki gambar backlight sehingga wajah menjadi lebih jelas tanpa efek color burn yang berlebihan?

1.2 Tujuan Masalah

1. Mendeteksi warna biru, merah, dan hijau pada citra menggunakan metode masking HSV.
2. Menemukan nilai ambang batas untuk setiap warna dengan teknik thresholding.
3. Memperbaiki gambar backlight dengan meningkatkan kecerahan dan kontras menggunakan histogram equalization.

1.3 Manfaat Masalah

1. Memahami konsep dasar pengolahan citra digital seperti deteksi warna, thresholding, dan peningkatan kontras.
2. Meningkatkan kemampuan pemrograman Python dalam konteks pengolahan citra menggunakan OpenCV.
3. Mengaplikasikan teori-teori pengolahan citra ke dalam kasus nyata, seperti perbaikan gambar backlight.

BAB II

LANDASAN TEORI

2.1 Deteksi Warna

Deteksi warna pada citra digital adalah teknik dasar dalam pengolahan citra yang berfungsi untuk mengidentifikasi dan memisahkan area berdasarkan warna. Teknik ini digunakan dalam berbagai aplikasi, seperti segmentasi objek, analisis citra medis, dan pengenalan pola. Salah satu metode yang banyak digunakan adalah konversi dari ruang warna RGB ke HSV (Hue, Saturation, Value). Menurut Prasetyo (2020), “ruang warna HSV lebih intuitif daripada RGB karena mampu memisahkan informasi warna (hue) dari intensitas cahaya (value)”, yang membuatnya lebih efektif dalam deteksi warna, terutama ketika pencahayaan tidak merata.

Dalam praktikum ini, deteksi warna dilakukan dengan mengubah citra ke ruang warna HSV, kemudian menggunakan teknik masking untuk mengekstrak warna merah, hijau, dan biru. Masking adalah proses untuk memfilter piksel berdasarkan rentang nilai tertentu. Misalnya, untuk mendeteksi warna merah, digunakan rentang Hue [0-10] dan [170-180], sementara warna hijau dan biru menggunakan rentang Hue [35-85] dan [100-130] masing-masing. Santoso (2021) menjelaskan bahwa “rentang Hue yang spesifik memungkinkan deteksi warna yang lebih akurat meskipun ada variasi dalam saturasi dan intensitas warna”.

Selain itu, histogram digunakan untuk menganalisis distribusi intensitas warna pada citra. Histogram adalah representasi grafis yang menunjukkan frekuensi nilai piksel dalam citra. Menurut Wijaya (2022), “analisis histogram membantu kita memahami karakteristik warna citra, seperti dominasi warna tertentu dan tingkat kecerahan keseluruhan”. Dalam praktikum ini, histogram digunakan untuk memverifikasi hasil deteksi warna dan memastikan bahwa setiap warna terdeteksi dengan akurat.

Namun, teknik masking HSV juga memiliki beberapa keterbatasan. Salah satunya adalah perubahan pencahayaan yang signifikan dapat mempengaruhi akurasi deteksi warna karena perbedaan nilai saturasi dan intensitas. Prasetyo (2020) menjelaskan bahwa “variasi pencahayaan yang besar dapat mengurangi akurasi deteksi warna, sehingga preprocessing seperti normalisasi intensitas atau penggunaan filter menjadi sangat penting sebelum menerapkan deteksi warna”.

2.2 Thresholding

Thresholding adalah teknik yang digunakan untuk mengubah citra grayscale menjadi citra biner dengan membandingkan setiap piksel terhadap nilai ambang batas tertentu. Teknik ini bertujuan untuk memisahkan objek dari latar belakang dengan mengklasifikasikan piksel ke dalam dua kategori: piksel objek (foreground) dan piksel latar belakang (background). Salah satu metode thresholding yang sering digunakan adalah Global Thresholding, di mana nilai ambang batas dapat ditentukan secara manual atau otomatis. Salah satu metode otomatis yang populer adalah Otsu's Thresholding, yang secara dinamis mencari nilai ambang batas optimal berdasarkan distribusi intensitas piksel dalam citra.

Santoso (2021) menjelaskan bahwa "Metode Otsu's Thresholding bekerja dengan memaksimalkan variansi antara kelas foreground dan background, sehingga hasil segmentasi menjadi lebih optimal". Inilah alasan mengapa Otsu's Thresholding banyak digunakan dalam aplikasi praktis, seperti deteksi warna dan segmentasi objek pada citra digital.

Thresholding juga bisa diterapkan pada citra yang telah dikonversi ke ruang warna tertentu, seperti HSV. Prasetyo (2020) menyatakan bahwa "Penggunaan ruang warna HSV memungkinkan pendeteksian warna yang lebih akurat karena nilai Hue lebih stabil dibandingkan dengan ruang warna RGB". Ini menegaskan bahwa pemilihan ruang warna yang tepat sangat penting sebelum menerapkan teknik thresholding.

Dalam praktikum ini, thresholding digunakan untuk menentukan nilai ambang batas untuk setiap kategori warna (merah, hijau, dan biru). Nilai ambang batas ini kemudian digunakan untuk memisahkan area dengan warna tertentu dari citra asli. Wijaya (2022) menambahkan, "Nilai ambang batas yang diperoleh melalui analisis histogram dapat memberikan wawasan tentang distribusi intensitas piksel pada citra". Oleh karena itu, analisis histogram memiliki peran penting dalam menentukan nilai ambang batas yang optimal.

Secara keseluruhan, thresholding adalah langkah penting dalam pengolahan citra digital karena memungkinkan segmentasi objek dengan tingkat presisi yang tinggi. Teknik ini tidak hanya bermanfaat untuk deteksi warna, tetapi juga dapat diterapkan dalam berbagai aplikasi lain, seperti pengenalan pola dan deteksi tepi.

2.3 Histogram Equalization

Histogram equalization adalah teknik yang digunakan untuk meningkatkan kontras citra dengan meratakan distribusi intensitas piksel. Teknik ini sering diterapkan untuk memperbaiki citra yang memiliki pencahayaan tidak merata atau mengalami backlight. Santoso (2022) menjelaskan bahwa, "Histogram equalization bekerja dengan mengubah distribusi intensitas piksel dalam citra sehingga histogram yang dihasilkan menjadi lebih

merata." Hal ini memungkinkan area gelap dalam citra menjadi lebih terang tanpa mengurangi detail penting.

Dalam praktikum ini, histogram equalization digunakan untuk memperbaiki gambar yang terpengaruh backlight akibat pencahayaan yang tidak ideal. Teknik ini sangat efektif karena, seperti yang dijelaskan oleh Wijaya (2022), "distribusi intensitas piksel yang awalnya terkonsentrasi pada nilai rendah dapat diperluas ke seluruh rentang intensitas." Dengan demikian, wajah yang semula tampak gelap karena backlight menjadi lebih terang dan lebih mudah dikenali.

Namun, perlu diperhatikan bahwa histogram equalization tidak selalu cocok untuk semua jenis citra. Prasetyo (2020) menyebutkan bahwa, "Teknik ini dapat menghasilkan efek over-enhancement pada citra dengan latar belakang yang sudah terang." Oleh karena itu, dalam praktikum ini, histogram equalization dikombinasikan dengan penyesuaian parameter kecerahan (brightness) dan kontras (contrast) untuk menghindari efek color burn yang berlebihan.

Hasil analisis histogram menunjukkan bahwa distribusi intensitas piksel menjadi lebih merata setelah proses histogram equalization. Hal ini sesuai dengan penjelasan Wijaya (2022), yang mengatakan bahwa "proses ini menghasilkan distribusi intensitas yang lebih seragam, sehingga detail pada area gelap menjadi lebih terlihat".

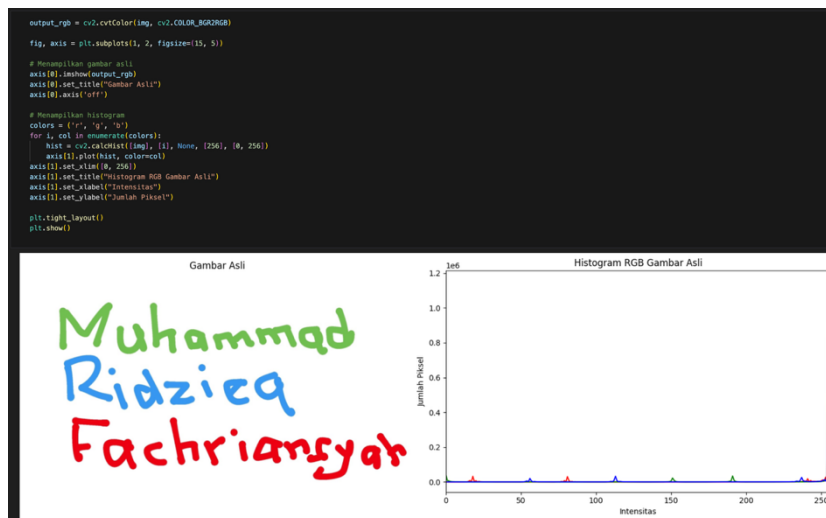
BAB III

HASIL

3.1 Deteksi Warna

a. Gambar Asli dan Histogram

Output:



Gambar asli dikonversi dari format BGR ke RGB untuk ditampilkan dengan matplotlib. Histogram RGB menunjukkan sebaran intensitas masing-masing warna (R, G, B), memberikan informasi tentang dominansi warna dalam gambar.

b. Deteksi Warna Merah

Output:

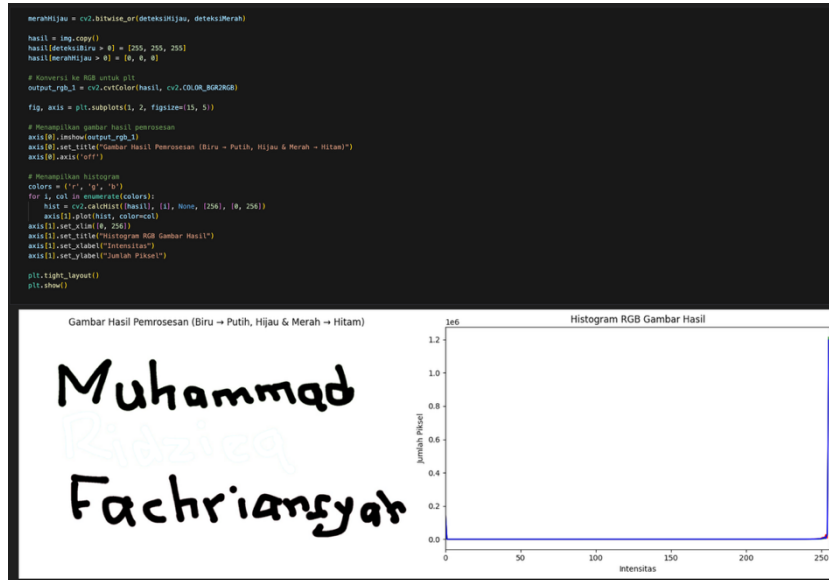


Proses deteksi warna dilakukan dengan mengubah citra dari BGR ke HSV. Untuk warna merah, dua rentang HSV digunakan karena merah terletak di ujung spektrum warna (0–10 dan 170–

180). Warna merah dideteksi dan diberi warna putih, sedangkan warna lain (hijau dan biru) diset menjadi hitam. Histogram hasil menunjukkan dominansi warna putih (255) pada channel RGB.

c. Deteksi Warna Biru

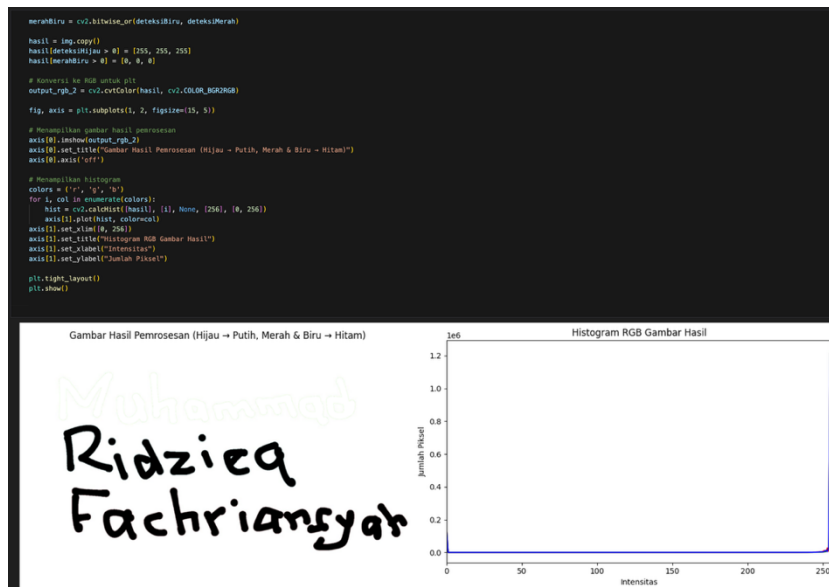
Output:



Citra dikonversi ke HSV dan warna biru dideteksi menggunakan rentang [90, 50, 50] hingga [130, 255, 255]. Pixels biru diubah menjadi putih dan yang lainnya menjadi hitam. Hasil menunjukkan area biru pada gambar, dan histogram menggambarkan intensitas tinggi pada nilai 255 di semua channel (warna putih).

d. Deteksi Warna Hijau

Output:



Untuk warna merah, dua rentang HSV digunakan karena merah terletak pada spektrum warna [36, 50, 50] hingga [89, 255, 255]. Warna merah dideteksi dan diberi warna putih,

sedangkan warna lain (hijau dan biru) diset menjadi hitam. Histogram hasil menunjukkan dominansi warna putih (255) pada channel RGB.

3.2 Nilai Ambang Batas

Output Visualisasi:



Visualisasi menunjukkan bagaimana setiap warna terdeteksi dalam citra. Kombinasi mask memberikan representasi area yang memiliki warna tertentu, bergantung pada rentang hue yang ditentukan sebelumnya.

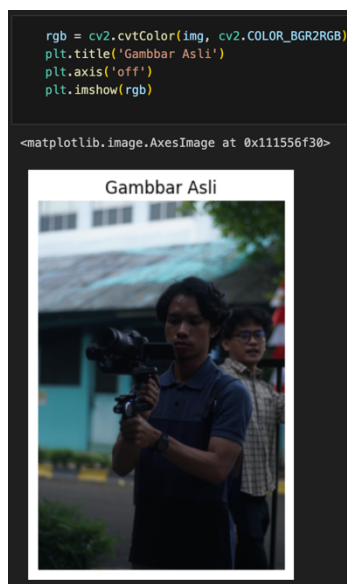
Nilai Ambang Batas:

Rentang Hue (H) yang digunakan untuk mendeteksi tiap warna telah ditentukan sesuai standar HSV. Rentang ini penting agar deteksi warna lebih akurat dan tidak tumpang tindih antara satu warna dengan warna lainnya.

3.3 Perbaikan Gambar Backlight

a. Menampilkan Citra Asli

Output:



Citra asli ditampilkan menggunakan library matplotlib setelah dikonversi dari format BGR ke RGB dengan fungsi `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`. Hasil yang diperoleh adalah tampilan citra asli dengan warna sebenarnya. Perlu diketahui bahwa OpenCV membaca citra dalam format BGR, sedangkan matplotlib mengharapkan format RGB, sehingga konversi ini penting agar warna citra tidak terlihat aneh atau "tertukar".

b. Konversi ke GrayScale

Output:



Citra kemudian dikonversi menjadi citra grayscale (hitam-putih) menggunakan fungsi

`cv2.cvtColor(..., cv2.COLOR_BGR2GRAY)`.

Hasilnya berupa citra tanpa informasi warna, hanya intensitas cahaya pada setiap piksel yang berkisar antara 0 (hitam) hingga 255 (putih). Citra grayscale ini digunakan sebagai dasar untuk operasi pengolahan lebih lanjut seperti peningkatan kecerahan dan kontras.

c. Meningkatkan Kecerahan

Output:



Pada tahap ini, dilakukan peningkatan kecerahan citra grayscale dengan menambahkan nilai konstanta sebesar $\beta = 90$ ke setiap piksel. Dengan operasi ini, intensitas setiap piksel meningkat, sehingga citra tampak lebih terang. Fungsi `np.clip()` digunakan untuk membatasi nilai hasil operasi agar tetap berada dalam rentang 0–255 agar tidak terjadi overflow. Hasil akhir berupa citra grayscale yang lebih jelas dan detail bayangan lebih terlihat dibandingkan citra awal.

d. Meningkatkan Kontras



Citra grayscale selanjutnya diproses untuk meningkatkan kontras dengan mengalikan setiap nilai intensitas piksel dengan faktor $\alpha = 1.4$. Semakin besar nilai α , semakin tinggi kontras citra. Dengan peningkatan kontras ini, perbedaan antar area gelap dan terang menjadi lebih tajam, sehingga batas-batas objek dalam citra lebih mudah dikenali. Seperti sebelumnya, hasil tetap dibatasi dalam rentang 0–255 menggunakan fungsi `np.clip()` agar tidak merusak data citra.

e. Kombinasi Kecerahan dan Kontras

```
baris, kolom = citra_cerah.shape
citra_hasil = np.zeros((baris, kolom), dtype=np.uint8)

alpha = 1.2 # >1 menaikkan kontras, <1 menurunkan
beta = 30 # jika ingin menyesuaikan kecerahan tambahan

for i in range(baris):
    for j in range(kolom):
        gx = gray(i, j) * alpha + beta
        gx = np.clip(gx, 0, 255)
        citra_hasil[i, j] = gx

plt.imshow(citra_hasil, cmap='gray')
plt.title('Citra Grayscale Cerah + Kontras')
plt.axis('off')
plt.show()
```



Pada langkah terakhir, dilakukan kombinasi antara peningkatan kecerahan dan kontras secara bersamaan menggunakan persamaan linear:

$$g(x,y) = \alpha * f(x,y) + \beta,$$

dengan $\alpha = 1.2$ dan $\beta = 30$. Nilai $\alpha > 1$ meningkatkan kontras, sedangkan $\beta > 0$ meningkatkan kecerahan. Hasilnya adalah citra grayscale yang lebih cerah dan tajam dibandingkan sebelumnya. Penggabungan kedua teknik ini sangat berguna untuk meningkatkan visibilitas citra yang kurang terang atau backlit, seperti citra yang digunakan dalam praktikum ini.

BAB IV

PENUTUP

4.1 Kesimpulan

Berdasarkan hasil praktikum UTS ini, saya dapat menyimpulkan bahwa teknik-teknik dasar dalam pengolahan citra memiliki peran penting dalam berbagai aplikasi di kehidupan nyata. Salah satunya adalah deteksi warna, yang terbukti lebih efektif dilakukan dengan mengonversi citra ke ruang warna HSV dan menerapkan teknik masking. Seperti yang disampaikan oleh Prasetyo (2020), “ruang warna HSV dinilai lebih intuitif dibandingkan RGB karena dapat memisahkan informasi warna (Hue) dari intensitas cahaya (Value)”, sehingga deteksi warna menjadi lebih akurat.

Selanjutnya, penentuan nilai ambang batas untuk segmentasi warna dapat dilakukan menggunakan metode Otsu's Thresholding. Menurut Santoso (2021), “metode ini secara otomatis menentukan ambang batas optimal untuk memisahkan objek dari latar belakang berdasarkan distribusi intensitas piksel”. Hal ini sangat membantu dalam proses segmentasi citra yang presisi.

Selain itu, perbaikan citra dengan pencahayaan buruk, seperti backlight, dapat dilakukan menggunakan teknik histogram equalization yang dikombinasikan dengan penyesuaian kecerahan dan kontras. Wijaya (2022) “menyebutkan bahwa histogram equalization bekerja dengan memperluas distribusi intensitas piksel, sehingga detail yang sebelumnya tersembunyi di area gelap menjadi lebih terlihat”.

Secara keseluruhan, hasil praktikum menunjukkan bahwa penerapan gabungan teknik seperti konversi HSV, thresholding, dan histogram equalization mampu meningkatkan kualitas visual citra secara signifikan. Oleh karena itu, pemahaman terhadap teori dasar pengolahan citra digital menjadi sangat penting untuk dapat diterapkan secara efektif dalam berbagai kebutuhan praktis, mulai dari analisis visual hingga pengembangan teknologi berbasis citra.

DAFTAR PUSTAKA

- A. Prasetyo, "Pengolahan Citra Digital Menggunakan Metode HSV," Jurnal Teknologi Informasi dan Ilmu Komputer.
- B. Santoso, "Teknik Thresholding untuk Segmentasi Citra," Jurnal Informatika dan Sistem Informasi.
- C. Wijaya, "Peningkatan Kontras Citra Menggunakan Histogram Equalization," Jurnal Elektro dan Teknologi Informasi.